

Практика 4. Введение в Haskell

осень 2026

Андрей Стоян, Илья Колегов

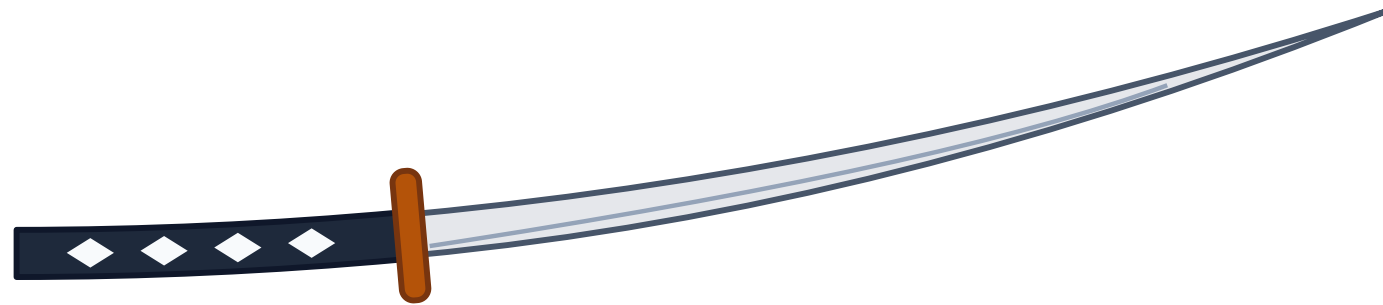
осень 2026

Летучка 4

Летучка 4 · 7 минут · Ваше ФИ: _____

1. Найдите наиболее общий тип терма $\lambda x y. y$: _____
2. Расставьте все скобки в типе: $(\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma \equiv$ _____
3. Сделайте один η -шаг: $\lambda x. a b x \rightsquigarrow_{\eta}$ _____
4. Чему равно значение выражения $(*2) . (+3) \$ 4$? _____

Почему Haskell



Haskell — катана

- Ничего лишнего
- Идеально демонстрирует концепции
- **NB** Совет: непонятное редуцируйте руками на простом примере, а чтобы осознать что-то — реализуйте это самостоятельно



Другие языки — швейцарский нож

- Перегружены под практические обстоятельства
- Удобно, но не видно сути

λ-исчисление ↔ Haskell

 **Устно:** Три недели мы писали на λ-исчислении. Как то же самое пишется в Haskell?

λ-исчисление	Haskell
абстракция $\lambda x. M$	<code>\x -> M</code>
аппликация $M\ N$, группируется влево	<code>M N</code>
тип $\alpha \rightarrow \beta$, стрелка группируется вправо	<code>a -> b</code>
I , K , K_*	<code>id, const, flip const</code>
композиция B	<code>(.)</code>
pair , fst , snd	<code>(,), fst, snd</code>
рекурсия через Y	функция вызывает себя по имени
η-шаг $\lambda x. M\ x \rightsquigarrow_{\eta} M$	бесточечный стиль

Рекурсия и аккумуляторы

Техника аккумулярующего параметра

✎ Какая сложность по памяти у рекурсивного факториала?

⚡ *Решение.*

Линейная: каждое умножение ждёт, пока вернётся рекурсивный вызов.

- Будем накапливать промежуточные результаты в параметре-аккумуляторе

✎ Прорецируйте пример `fac' 4`

```
fac' n = go 1 n
  where
    go acc n'
      | n' <= 1 = acc
      | otherwise = go (acc * n') (n' - 1)
```

Рекурсия против аккумулятора

Сначала рекурсия. Раскрываем `fac 4` по определению.

`fac 4`

ЦЕПОЧКА
РЕКУРСИЯ
`fac 4`

Рекурсия против аккумулятора

4 > 1, поэтому вызов превращается в умножение. Умножить пока нельзя: второй множитель ещё не вычислен.

4 * fac 3

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

→ 4 * fac 3

Рекурсия против аккумулятора

То же самое внутри: ещё одно умножение встало в очередь.

4 * (3 * **fac 2**)

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

↪ 4 * fac 3

↪ 4 * (3 * fac 2)

Рекурсия против аккумулятора

Выражение растёт с каждым вызовом: в нём хранится всё, что ещё предстоит умножить.

4 * (3 * (2 * **fac 1**))

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

→ 4 * fac 3

→ 4 * (3 * fac 2)

→ 4 * (3 * (2 * fac 1))

Рекурсия против аккумулятора

`fac 1` отвечает 1 без рекурсии. Только теперь умножениям есть что умножать.

$4 * (3 * (2 * 1))$

ЦЕПОЧКА

РЕКУРСИЯ

`fac 4`

→ `4 * fac 3`

→ `4 * (3 * fac 2)`

→ `4 * (3 * (2 * fac 1))`

→ `4 * (3 * (2 * 1))`

Рекурсия против аккумулятора

Очередь умножений разобрана. Память линейная: длина выражения росла вместе с числом вызовов.

24

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

⇒ 4 * fac 3

⇒ 4 * (3 * fac 2)

⇒ 4 * (3 * (2 * fac 1))

⇒ 4 * (3 * (2 * 1))

⇒ 24

Рекурсия против аккумулятора

Теперь аккумулятор. Начинаем с того же вызова, только функция другая.

fac' 4

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

⇒ 4 * fac 3

⇒ 4 * (3 * fac 2)

⇒ 4 * (3 * (2 * fac 1))

⇒ 4 * (3 * (2 * 1))

⇒ 24

АККУМУЛЯТОР

fac' 4

Рекурсия против аккумулятора

`fac'` 4 раскрывается по определению в `go 1 4`: промежуточный результат 1, счётчик 4.

`go` 1 4

ЦЕПОЧКА

РЕКУРСИЯ

`fac 4`

⇒ `4 * fac 3`

⇒ `4 * (3 * fac 2)`

⇒ `4 * (3 * (2 * fac 1))`

⇒ `4 * (3 * (2 * 1))`

⇒ 24

АККУМУЛЯТОР

`fac' 4`

⇒ `go 1 4`

Рекурсия против аккумулятора

4 > 1: умножаем аккумулятор на счётчик сразу, счётчик уменьшаем. Ждать нечего.

go 4 3

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

⇒ 4 * fac 3

⇒ 4 * (3 * fac 2)

⇒ 4 * (3 * (2 * fac 1))

⇒ 4 * (3 * (2 * 1))

⇒ 24

АККУМУЛЯТОР

fac' 4

⇒ go 1 4

⇒ go 4 3

Рекурсия против аккумулятора

Выражение не растёт: всё, что уже посчитано, лежит в аккумуляторе одним числом.

go 12 2

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

⇒ 4 * fac 3

⇒ 4 * (3 * fac 2)

⇒ 4 * (3 * (2 * fac 1))

⇒ 4 * (3 * (2 * 1))

⇒ 24

АККУМУЛЯТОР

fac' 4

⇒ go 1 4

⇒ go 4 3

⇒ go 12 2

Рекурсия против аккумулятора

Счётчик дошёл до единицы — следующим шагом **go** вернёт аккумулятор.

go 24 1

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

→ 4 * fac 3

→ 4 * (3 * fac 2)

→ 4 * (3 * (2 * fac 1))

→ 4 * (3 * (2 * 1))

→ 24

АККУМУЛЯТОР

fac' 4

→ go 1 4

→ go 4 3

→ go 12 2

→ go 24 1

Рекурсия против аккумулятора

Сравните две цепочки справа: первая расширяется с каждым шагом, вторая — нет.

24

ЦЕПОЧКА

РЕКУРСИЯ

fac 4

⇒ 4 * fac 3

⇒ 4 * (3 * fac 2)

⇒ 4 * (3 * (2 * fac 1))

⇒ 4 * (3 * (2 * 1))

⇒ 24

АККУМУЛЯТОР

fac' 4

⇒ go 1 4

⇒ go 4 3

⇒ go 12 2

⇒ go 24 1

⇒ 24

Злополучный Фибоначчи

- Реализация по определению имеет экспоненциальную сложность

✎ Заполните пропуски, чтобы `fib` вычислял n -е число Фибоначчи за линейное время

```
fib :: Int -> Int
fib n = go 1 1 n
  where
    go prev curr n'
      | n' <= 1 = -----
      | otherwise = go ----- (n' - 1)
```

⚡ **Решение.**

```
go prev curr n'
  | n' <= 1 = prev
  | otherwise = go curr (prev + curr) (n' - 1)
```

Операторы и композиция

Расставьте скобки

```
infixr 9 .  
infixr 8 ^  
infixl 7 *  
infixl 6 +, -  
infixr 0 $
```

-- применение функции сильнее любого оператора и группируется влево

 Расставьте все скобки:

(a) $2 + 3 * 4 ^ 2$

(б) $f\ x + g\ y * 2$

(в) $f\ .\ g\ \$\ x + 1$

 **Решение.**

(a) $2 + (3 * (4 ^ 2))$

(б) $(f\ x) + ((g\ y) * 2)$

(в) $(f\ .\ g)\ \$\ (x + 1)$

Конструирование реализаций

```
($) :: (a -> b) -> a -> b
(&) :: a -> (a -> b) -> b
(.) :: (b -> c) -> (a -> b) -> a -> c
f :: Int -> a -> a
g :: Int -> Double
h :: Int -> Double -> Int
```

✎ Перепишите `example x = f 1 (g (h 2 x))`:

(а) через доллар; (б) через композицию; (в) в бесточечном стиле; (г) через `&`; (д) через `let`

⚡ Решение.

```
example x = f 1 $ g $ h 2 x           -- (а)
example x = (f 1 . g . h 2) x         -- (б)
example   = f 1 . g . h 2             -- (в): n-шаг
example x = x & h 2 & g & f 1           -- (г): слева направо, как конвейер
example x = let x1 = h 2 x in          -- (д): именуем промежуточные вычисления
              let x2 = g x1 in
              f 1 x2
```

Полиморфные функции

Предскажите тип

- Типы с маленькой буквы — типовые переменные
- Могут принимать разные значения в зависимости от контекста

```
 ghci> :t id
```

```
 ghci> :t id "something here"
```

⚡ Решение.

```
id :: a -> a
```

⚡ Решение.

```
id "something here" :: String
```

Предскажите тип

```
const :: a -> b -> a
flip  :: (a -> b -> c) -> b -> a -> c
```

```
ghci> :t const id
```

```
ghci> :t flip const
```

⚡ *Решение.*

```
const id :: b -> a -> a
```

⚡ *Решение.*

`flip const :: b -> c -> c` — тот же тип: буквы компилятор выбирает сам, их можно переименовать.

Это тип **K**_{*} из летучки

Читаем сообщения GHC

```
ghci> :t const True 'a' 'b'  
error: [GHC-83865]
```

- Couldn't match expected type 'Char -> t' with actual type 'Bool'
- In the first argument of 'const', namely 'True'
In the expression: const True 'a' 'b'

 **Устно:** Переведите сообщение: какой тип компилятор ожидал, какой получил и откуда взялось ожидание?

⚡ *Решение.*

Аппликация группируется влево: `((const True) 'a') 'b'`. Результат `const True 'a'` имеет тип первого аргумента, а его применяют к `'b'` — значит, первый аргумент обязан быть функцией `Char -> t`. Получен `True :: Bool`.

Типовые дыры

В ПЗ мы населяли тип композиции, ставя дырку $\square$ и подписывая её тип. В Haskell дырка — это `_`, и её тип подписывает компилятор.

```
compose :: (b -> c) -> (a -> b) -> a -> c
compose f g x = _
```

- Found hole: `_ :: c`
- Relevant bindings include
 - `x :: a`
 - `g :: a -> b`
 - `f :: b -> c`

 **Устно:** Чем в терминах ПЗ является список «Relevant bindings»?

 **Решение.**

Контекстом типизации: $\Gamma \equiv f^{\beta \rightarrow \gamma}, g^{\alpha \rightarrow \beta}, x^{\alpha}$

Населяем тип по шагам

```
compose :: (b -> c) -> (a -> b) -> a -> c
-- контекст: f :: b -> c,   g :: a -> b,   x :: a
```

пишем

```
compose f g x = _
compose f g x = f _
compose f g x = f (g _)
compose f g x = f (g x)
```

компилятор отвечает

```
Found hole: _ :: c
Found hole: _ :: b
Found hole: _ :: a, подходит x
Ok, one module loaded.
```

- ➡ Тип подсказывает реализацию: программу можно вести от сигнатуры, спрашивая компилятор на каждом шаге

Что вы теперь умеете

- Читать Haskell как λ -исчисление с типами: $\lambda x. M, a \rightarrow b, \text{const}$ — это **K**
- Судить о рекурсивной функции: завершается ли она и что копится, пока она считает
- Писать рекурсию с аккумулятором
- Собирать функции через $\$$ и $.$ и убирать точку η -шагом
- Предсказывать тип выражения и переводить сообщение компилятора
- Строить функцию по её типу, спрашивая компилятор дырками

В следующих сериях: сегодня все данные были числами и парами — дальше соберём собственные типы данных

Что нового в инструментах

- Домашка 4 живёт в знакомом шаблоне, но `make repl` теперь открывает GHCi с вашим кодом на Haskell
- В GHCi: `:t` — тип выражения, `:i` — сведения об имени и приоритет оператора, `:r` — перечитать файл после правки
- `make check` перед тестами запускает линтер `hlint`
- **NB** Линтер на этом курсе обязателен: его подсказки учат идиомам языка
- **NB** Форматтер на этом курсе запрещён: он переписывает файл целиком, а нам нужны минимальные диффы

Материалы

Компилятор GHC

- Единственный компилятор Haskell
- Язык развивается в основном расширениями компилятора
 - Подключение прагмой в начале файла `{-# LANGUAGE ... #-}`
 - Или ключами компиляции (в конфигах проекта)
 - Добавляют синтаксический сахар (`LambdaCase`, `ImplicitParams`)
 - Обогащают систему типов (`RankNTypes`)
 - Добавляют фичи (`MultiParamTypeClasses`, `GADTs`)
 - Добавляют экспериментальные фичи (`LinearTypes`)
- https://ghc.gitlab.haskell.org/ghc/doc/users_guide/index.html

Системы сборки

- По конфигурации проекта правильно вызывают компилятор

Cabal — Common Architecture for Building Applications and Libraries

- Система сборки + библиотека; на этом курсе пользуемся ей
- Компилятор нужной версии ставит `ghcup`, версия записана в `cabal.project`
- `cabal build` — собрать проект
- `cabal test` — запустить тесты
- `cabal repl` — REPL с загруженным проектом
- В домашках эти команды завернуты в `make build`, `make check`, `make repl`

Stack — надстройка над cabal, которая сама скачивает компилятор; встретится в чужих проектах

REPL, GHCi

REPL (read-eval-print-loop)

Интерактивное окружение, исполняющее вводимые в него высказывания и тут же демонстрирующее результат.

- Компилятор работает долго, а GHCi — быстро
- Пишем код, загружаем в интерпретатор: `make repl`
- Основные возможности
 - Исполнить выражение
 - `:r` (reload) — перезагрузить интерпретатор
 - `:t` (type) — посмотреть тип выражения
 - `:i` (info) — посмотреть документацию функции, приоритет оператора
 - `:set` — установить расширения языка и флаги компиляции

Другие тулы

- Линтер (`hlint`) помогает находить ошибки в программах
- Очень помогает в обучении
- **NB** На этом курсе обязателен к использованию!
- Форматтер автоматически форматирует код: `ormolu`, `fourmolu`
- Обязателен к использованию во многих языках: экономит время и повышает консистентность кода
- В Haskell не всегда удовлетворительно работает
- **NB** На этом курсе запрещён: он переписывает файл целиком, а нам нужны минимальные диффы

- LSP — Language Server Protocol — это протокол взаимодействия между движками IDE и текстовыми редакторами
- Есть language server для большинства языков
- [Есть для Haskell](#), работает *только* для той же версии ghc
- Большинство редакторов кода поддерживает LSP и с его помощью почти превращается в IDE

IDE для Haskell на примере vscode

- Ставите `haskell-language-server` через `ghcup` — он подберёт сборку под ваш компилятор
- Устанавливаете в систему `hlint`
- Ставите плагины: [Haskell \(может поставить и остальные\)](#), [Haskell Syntax](#)

Полиморфные функции *

- * То, что типизация остаётся валидной после подстановки в тип — теорема метатеории

Лемма подстановки типа для $\lambda_{\rightarrow}$

$$\begin{array}{ll} \Gamma \vdash M : A \Rightarrow [\alpha \mapsto B] \Gamma \vdash & M : [\alpha \mapsto B] A \quad (\lambda_{\rightarrow} \text{ по Карри}) \\ \Gamma \vdash M : A \Rightarrow [\alpha \mapsto B] \Gamma \vdash [\alpha \mapsto B] M : [\alpha \mapsto B] A & (\lambda_{\rightarrow} \text{ по Чёрчу}) \end{array}$$

- Если мы в языке разрешим приписывать типы термам, мы потеряем гибкость:

$$\begin{array}{l} \mathbf{K} : \alpha \rightarrow \beta \rightarrow \alpha \\ \mathbf{K} \ 1 : \text{cannot match } \alpha \text{ with Int} \end{array}$$

- Введём в теорию абстракцию на типах и аппликацию термов к типам!

$$\begin{array}{l} \mathbf{K} : \forall \alpha \ \beta. \alpha \rightarrow \beta \rightarrow \alpha \\ \mathbf{K} \ @\text{Int} : \forall \beta. \text{Int} \rightarrow \beta \rightarrow \text{Int} \\ \mathbf{K} \ @\text{Int} \ @\text{Float} \ 1 : \text{Float} \rightarrow \text{Int} \end{array}$$

- Теперь выбор конкретного типа происходит на стороне пользователя функции
- Такие функции называются **полиморфными** — умеют работать с различными типами

Полиморфные функции в Haskell *

- Haskell неявно приписывает `forall` по всем типовым переменным:

```
ghci> :set -fprint-explicit-foralls
ghci> :t const
const :: forall a b. a -> b -> a
```

- Типовые переменные всегда начинаются с маленькой буквы (`a`), в отличие от типовых констант (`Int` — конкретный тип)
- Типовые применения компилятор выставляет автоматически неявно
- Иногда бывает полезно типовые применения указывать вручную (`TypeApplications`)

- ```
ghci> :t const @Int
const @Int :: forall b. Int -> b -> Int
```

- В C++-like языках синтаксис примерно такой:

```
fun <A, B> const(x: A, y: B): A
const<Int, Float>(1, 2)
```

# Что посмотреть в транспорте

- [Vitaly Bragilevsky - The clear path to Haskell complexities](#)
- [Podlodka #211 – Haskell | Александр Вершилов](#)
- [Property-based testing в Haskell](#) — так тестируются наши домашки

# Полезные ссылки

- [Hoogle](#) — типизированный поиск по документации библиотек
- [Отладка принтами: Debug.Trace](#)
- [GHC docs](#) — про компилятор и расширения языка
- [Typeclassopedia](#)
- [ghcup](#) и [Cabal User Guide](#)
- [The Haskell Execution Model](#)
- [I know kung fu: learning STG by example](#)

# Серьёзные материалы

- [Will Kurt «Get Programming with Haskell»](#)
- Две части курса Дениса Николаевича на степике: [раз](#) и [два](#)
- [\(youtube\) Лекции Дениса Николаевича на МКН](#)
- [Vitaly Bragilevsky «Haskell in depth»](#)
- [\(youtube\) Компилятор GHC языка Haskell: теория языков программирования в работе](#)
- [\(youtube\) Continuation-passing style, defunctionalization, and associativity.](#)